Nomad

A Functional Language with Borrow Checking

Walid Menou

Abstract

This project implements an interpreter for a functional programming language extended with mutability and borrow checking, investigating affine type systems for resource management. Borrow checking, famously used in Rust, provides strong compile-time guarantees about how values are used and mutated without relying on a garbage collector. A second objective is to explore expressiveness in language design by implementing features such as list comprehensions, a pipe operator, algebraic data types, pattern matching, and Hindley-Milner type inference. The developer should not have to manually annotate types or write redundant code, and should be able to express ideas concisely while still benefiting from strong compile-time guarantees. This report defines the language as it stands. It gives the grammar, the typing rules and the evaluation rules for every construct the interpreter supports, and it records the work still to come.

Contents

1 Introduction

Nomad is a small, strict, statically typed functional language in the ML family. A program is a list of top-level bindings and expressions. The interpreter reads the whole program, gives every expression a type, and only then runs it. No Nomad program carries a type annotation, because the checker works out every type on its own.

The aim is a language that solves programming challenges quickly. Someone working through a contest problem or an exercise set wants to write the idea down and get an answer. That person should not have to declare types twice, wrap input in a channel object, or spell out a fold that one line of set-builder notation would say. Nomad tries to keep the guarantees a strong type system gives while asking for none of the paperwork.

This report tracks the implementation rather than describing a finished language. It defines what the interpreter does today: Section 2 gives the grammar, Section 3 takes each construct in turn and states its typing rule and its evaluation rule, Section 4 does the same for comprehensions, and Section 5 explains how the checker finds types without being told them. The parser is a library of combinators in the style Hutton and Meijer set out [1].

Borrow checking is the next piece of work and the reason the project exists. It has no rules here yet. Section 6 lists what comes after it.

1.1 A first program

The program below sorts a list. It uses most of what the language offers: recursive bindings, functions of several arguments, list patterns and comprehensions.

```
let rec concat l1 l2 =
  match l1 with
  [] -> l2
  | x :: xs -> x :: concat xs l2

let rec quicksort lst =
  match lst with
  [] -> []
  | x :: xs ->
      concat
        (quicksort [y | y <- xs, y <= x])
        (x :: quicksort [y | y <- xs, y > x])

let result = quicksort [5; 2; 9; 1; 5; 6]
```

It prints `[1; 2; 5; 5; 6; 9]`. Nothing in it names a type, and the checker still rejects it if any part does not fit. The two comprehensions are doing the work that two auxiliary filtering functions would otherwise have to do.

2 Syntax

2.1 Grammar

A program is a sequence of statements. Each statement either binds a name or evaluates an expression. A binding may take parameters, which is shorthand for a function that returns a function.

$$
\begin{array}{rcl}
\mathit{prog} & ::= & \mathit{stmt}^* \\
\mathit{stmt} & ::= & \textbf{let}\ x\ y_1 \ldots y_k = e \quad | \quad \textbf{let rec}\ f\ y_1 \ldots y_k = e \quad | \quad e \\
e & ::= & \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 \\
 & | & \textbf{fun}\ y_1 \ldots y_k \rightarrow e \\
 & | & \textbf{let}\ x\ y_1 \ldots y_k = e_1\ \textbf{in}\ e_2 \quad | \quad \textbf{let rec}\ f\ y_1 \ldots y_k = e_1\ \textbf{in}\ e_2 \\
 & | & \textbf{match}\ e\ \textbf{with}\ p_1 \rightarrow e_1 \mid \cdots \mid p_n \rightarrow e_n \\
 & | & e_1 \,||\, e_2 \quad | \quad e_1 \,\&\&\, e_2 \quad | \quad e_1\ \mathit{relop}\ e_2 \\
 & | & e_1 \,|\!> e_2 \quad | \quad e_1 :: e_2 \\
 & | & e_1\ \mathit{addop}\ e_2 \quad | \quad e_1\ \mathit{mulop}\ e_2 \quad | \quad -e \quad | \quad e_1\ e_2 \\
 & | & x \quad | \quad n \quad | \quad b \quad | \quad s \quad | \quad () \quad | \quad (\,e\,) \\
 & | & [\,e_1; \ldots; e_n\,] \quad | \quad [\,e_1 .. e_2\,] \quad | \quad [\,e \mid q_1, \ldots, q_k\,] \\
q & ::= & p \leftarrow e \quad | \quad e \quad | \quad \textbf{let}\ x\ y_1 \ldots y_k = e \\
p & ::= & \_ \quad | \quad x \quad | \quad n \quad | \quad b \quad | \quad s \quad | \quad [\,] \quad | \quad p_1 :: p_2 \\
\mathit{relop} & ::= & < \mid <= \mid > \mid >= \mid = \mid <> \\
\mathit{addop} & ::= & + \mid - \qquad \mathit{mulop} ::= * \mid / \mid \% \qquad b ::= \textbf{true} \mid \textbf{false}
\end{array}
$$

Level	Operators	Groups	Note
1 (loosest)	`||`	left	
2	`&&`	left	
3	`< <= > >= = <>`	chained	Section 3.5
4	`|>`	left	Pipe
5	`::`	right	Cons
6	`+ -`	left	
7	`* / %`	left	
8	`-` (prefix)	prefix	Negation
9 (tightest)	application	left	

Table 1: Operator precedence, loosest first.

The four keyword forms `if`, `fun`, `let` and `match` run as far to the right as they can, which the table cannot show. In `if c then a else b + 1` the `else` branch is `b + 1`, and parentheses cut a form short.

2.2 Layout, comments and lexical rules

A statement ends where the next line begins in the first column. Whitespace inside a statement crosses a newline only when the line that follows is indented, so a continuation line must be indented and a top-level binding must not be. Two semicolons end a statement outright, which is the way to put more than one on a line, and they are optional everywhere else.

```
let a = 1 -- one statement
let b = -- another, continued below
  a + 1
let c = 2 ;; let d = 3 -- two on one line
```

A comment runs from `--` to the end of the line and counts as whitespace. Blank lines and comment lines are looked past when deciding whether the next line is indented, so a comment in the first column does not end the statement above it.

An identifier starts with a letter and continues with letters, digits and underscores. The words `let`, `in`, `fun`, `if`, `then`, `else`, `match` and `with` are reserved.

An integer literal carries no sign, since a leading minus is the prefix operator of level 8. This makes spacing matter in one place, and the rule is the one Haskell uses: `1 - 2` and `1 -2` are both subtraction, `1 -2` is `1` followed by a comment, and a negative right operand needs a space, as in `1 - -2`. A pattern keeps the signed literal, since a sign in a pattern cannot be an operator.

A string literal is any run of characters between two double quotes. There are no escape sequences.

3 Semantics

Two judgements run through this section. The first, $\Gamma \vdash e : \tau$, says that in the typing environment Γ the expression e has type τ. The second, $\rho \vdash e \Downarrow v$, says that in the value environment ρ the expression e evaluates to the value v. Both environments are finite maps written as lists of bindings, and a later binding hides an earlier one of the same name. Evaluation is big-step: it relates an expression straight to its answer, and an expression that goes wrong, such as a division by zero, simply has no derivation.

A type is a base type, a function type, a list type or a type variable. Types never appear in Nomad source and show up only in what the checker reports.

$$\tau ::= \mathsf{int} \mid \mathsf{bool} \mid \mathsf{string} \mid \mathsf{unit} \mid \tau_1 \to \tau_2 \mid \tau\ \mathsf{list} \mid \alpha$$

Evaluation produces values. A closure pairs a function with the environment it was written in, and a recursive closure also records the name the function calls itself by.

$$v ::= n \mid b \mid s \mid () \mid [v_1, \ldots, v_n] \mid \langle \mathbf{fun}\ x \to e,\ \rho \rangle \mid \langle f,\ \mathbf{fun}\ x \to e,\ \rho \rangle$$

3.1 Literals

The language has four kinds of literal: integers, booleans, strings and the unit value. Integers are machine integers, 63 bits wide on a 64-bit host, and they wrap on overflow. The unit value `()` is the only value of type `unit` and stands for a result that carries no information. A literal has the type its shape says it has, and it is already a value.

$$\frac{}{\Gamma \vdash n : \mathsf{int}}\ \textsc{T-Int} \qquad \frac{}{\Gamma \vdash b : \mathsf{bool}}\ \textsc{T-Bool} \qquad \frac{}{\Gamma \vdash s : \mathsf{string}}\ \textsc{T-Str} \qquad \frac{}{\Gamma \vdash () : \mathsf{unit}}\ \textsc{T-Unit}$$

$$\frac{}{\rho \vdash c \Downarrow c}\ \textsc{E-Lit}$$

Here c stands for any of n, b, s and $()$.

3.2 Variables

A variable stands for whatever the nearest enclosing binder gave it. There is no way to declare a name without giving it a value, and a name the environment does not mention is an error the checker reports before the program runs.

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ \textsc{T-Var} \qquad \frac{\rho(x) = v}{\rho \vdash x \Downarrow v}\ \textsc{E-Var}$$

Rule T-Var is stated for a plain type. Section 5.2 refines it, because the environment in fact records a scheme and each use takes a fresh copy of it.

3.3 Arithmetic

The five arithmetic operators take integers and give integers. Division truncates towards zero, so `7 / 2` is `3` and `-7 / 2` is `-3`, and the remainder takes the sign of the left operand, so `-7 % 3` is `-1` while `7 % -3` is `1`. A zero right operand of either has no value and stops the program. Prefix minus stands for subtraction from zero and needs no rule of its own.

$$\frac{\Gamma \vdash e_1 : \mathsf{int} \quad \Gamma \vdash e_2 : \mathsf{int} \quad \oplus \in \{+, -, *, /, \%\}}{\Gamma \vdash e_1 \oplus e_2 : \mathsf{int}}\ \textsc{T-Arith}$$

$$\frac{\rho \vdash e_1 \Downarrow n_1 \quad \rho \vdash e_2 \Downarrow n_2 \quad n_1 [\![\oplus]\!] n_2 = n}{\rho \vdash e_1 \oplus e_2 \Downarrow n}\ \textsc{E-Arith}$$

The metafunction $[\![\oplus]\!]$ is the operation on integers itself. It is undefined when $\oplus$ is division or remainder and n_2 is zero. The left operand is evaluated first.

3.4 Comparison and equality

The four relational operators compare integers. Equality and inequality compare two values of any one type, and the rule places no constraint on that type beyond the two sides agreeing.

$$\frac{\Gamma \vdash e_1 : \mathsf{int} \quad \Gamma \vdash e_2 : \mathsf{int} \quad \otimes \in \{<, \leq, >, \geq\}}{\Gamma \vdash e_1 \otimes e_2 : \mathsf{bool}}\ \textsc{T-Cmp} \qquad \frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau \quad \odot \in \{=, \neq\}}{\Gamma \vdash e_1 \odot e_2 : \mathsf{bool}}\ \textsc{T-Eq}$$

$$\frac{\rho \vdash e_1 \Downarrow n_1 \quad \rho \vdash e_2 \Downarrow n_2 \quad n_1 [\![\otimes]\!] n_2 = b}{\rho \vdash e_1 \otimes e_2 \Downarrow b}\ \textsc{E-Cmp} \qquad \frac{\rho \vdash e_1 \Downarrow v_1 \quad \rho \vdash e_2 \Downarrow v_2 \quad v_1 \equiv v_2 = b}{\rho \vdash e_1 = e_2 \Downarrow b}\ \textsc{E-Eq}$$

The relation $v_1 \equiv v_2$ is structural equality, so two values of the same type are equal when they are built the same way. It is undefined on closures, since reading two functions cannot decide whether they agree, and a function nested inside a list is caught as well. Making that a static error rather than a dynamic one would need a notion of equality type, which the language does not have.

3.5 Chained comparison

Mathematics writes $0 \leq x < 5$ and means both parts at once. Most languages read that as two comparisons in a row and produce nonsense. Nomad reads it the way mathematics does: a run of relational operators expands into a conjunction in which each operand is compared with both of its neighbours.

$$e_0 \otimes_1 e_1 \otimes_2 e_2 \ \cdots \ \otimes_k e_k \quad \rightsquigarrow \quad (e_0 \otimes_1 e_1) \mathbin{\&\&} (e_1 \otimes_2 e_2) \mathbin{\&\&} \cdots \mathbin{\&\&} (e_{k-1} \otimes_k e_k)$$

The conjunctions group to the left, and a single comparison is the case $k = 1$, which expands to itself. The parser performs the expansion, so no rule of the type system or the evaluator mentions chains. Equality sits on this level too, so `a = b = c` asks whether all three agree rather than comparing a boolean to `c`.

Each interior operand appears twice in the expansion. Nothing in Nomad has a side effect, so this changes only how much work the program does, but a language with input or mutation would have to bind the operands first.

3.6 Conjunction and disjunction

Both connectives take booleans and give a boolean, and both look at the right operand only when the left has not already settled the answer. A guard can therefore protect the expression after it, which is what makes `i < n && a i = 0` safe to write.

$$\frac{\Gamma \vdash e_1 : \mathsf{bool} \quad \Gamma \vdash e_2 : \mathsf{bool}}{\Gamma \vdash e_1 \mathbin{\&\&} e_2 : \mathsf{bool}}\ \textsc{T-And} \qquad \frac{\Gamma \vdash e_1 : \mathsf{bool} \quad \Gamma \vdash e_2 : \mathsf{bool}}{\Gamma \vdash e_1 \parallel e_2 : \mathsf{bool}}\ \textsc{T-Or}$$

$$\frac{\rho \vdash e_1 \Downarrow \textbf{false}}{\rho \vdash e_1 \mathbin{\&\&} e_2 \Downarrow \textbf{false}}\ \text{E-AndF} \qquad \frac{\rho \vdash e_1 \Downarrow \textbf{true} \quad \rho \vdash e_2 \Downarrow b}{\rho \vdash e_1 \mathbin{\&\&} e_2 \Downarrow b}\ \text{E-AndT}$$

$$\frac{\rho \vdash e_1 \Downarrow \textbf{true}}{\rho \vdash e_1 \mathbin{\|} e_2 \Downarrow \textbf{true}}\ \text{E-OrT} \qquad \frac{\rho \vdash e_1 \Downarrow \textbf{false} \quad \rho \vdash e_2 \Downarrow b}{\rho \vdash e_1 \mathbin{\|} e_2 \Downarrow b}\ \text{E-OrF}$$

3.7 Conditional

A conditional picks one of two branches. It is an expression, so it always produces a value and the `else` branch is never optional. Both branches must have the same type, since the checker cannot know which one will run, and only the branch taken is evaluated.

$$\frac{\Gamma \vdash e_1 : \mathsf{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \textbf{if } e_1 \textbf{ then } e_2 \textbf{ else } e_3 : \tau}\ \text{T-If}$$

$$\frac{\rho \vdash e_1 \Downarrow \textbf{true} \quad \rho \vdash e_2 \Downarrow v}{\rho \vdash \textbf{if } e_1 \textbf{ then } e_2 \textbf{ else } e_3 \Downarrow v}\ \text{E-IfT} \qquad \frac{\rho \vdash e_1 \Downarrow \textbf{false} \quad \rho \vdash e_3 \Downarrow v}{\rho \vdash \textbf{if } e_1 \textbf{ then } e_2 \textbf{ else } e_3 \Downarrow v}\ \text{E-IfF}$$

3.8 Functions and application

A function takes one argument. Writing several parameters is shorthand: `fun x y -> e` is `fun x -> fun y -> e`, and `let f x y = e` binds `f` to the same thing. Applying such a function to fewer arguments than it expects gives back a function that remembers the ones supplied so far. The parameters carry no annotation, and the checker works out their types from how the body uses them.

A function evaluates to a closure at once, and the body waits until the function is applied. The closure keeps the environment the function was written in, which is what makes the language lexically scoped: a free variable in the body means what it meant where the function appears in the source, not what it happens to mean where the function is called.

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \textbf{fun } x \to e : \tau_1 \to \tau_2}\ \text{T-Fun} \qquad \frac{}{\rho \vdash \textbf{fun } x \to e \Downarrow \langle \textbf{fun } x \to e,\ \rho \rangle}\ \text{E-Fun}$$

Application is call by value. The evaluator reduces the function, then the argument, and runs the body in the closure's environment extended with the argument.

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}\ \text{T-App}$$

$$\frac{\rho \vdash e_1 \Downarrow \langle \textbf{fun}\ x \to e,\ \rho' \rangle \quad \rho \vdash e_2 \Downarrow v_2 \quad \rho', x \mapsto v_2 \vdash e \Downarrow v}{\rho \vdash e_1\ e_2 \Downarrow v}\ \text{E-App}$$

A recursive closure works the same way, except that it puts its own name back in scope before running the body. That is what lets the function call itself, and it needs no mutable cell.

$$\frac{\rho \vdash e_1 \Downarrow \langle f,\ \textbf{fun}\ x \to e,\ \rho' \rangle \quad \rho \vdash e_2 \Downarrow v_2 \quad \rho', f \mapsto \langle f,\ \textbf{fun}\ x \to e,\ \rho' \rangle,\ x \mapsto v_2 \vdash e \Downarrow v}{\rho \vdash e_1\ e_2 \Downarrow v}\ \text{E-AppRec}$$

3.9 Local and recursive bindings

A `let` names the value of one expression and makes that name available in another. It is an expression, so its value is the value of its body. The bound expression runs first and runs once, whether or not the body uses it.

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \textbf{let}\ x = e_1\ \textbf{in}\ e_2 : \tau_2}\ \text{T-Let} \qquad \frac{\rho \vdash e_1 \Downarrow v_1 \quad \rho, x \mapsto v_1 \vdash e_2 \Downarrow v}{\rho \vdash \textbf{let}\ x = e_1\ \textbf{in}\ e_2 \Downarrow v}\ \text{E-Let}$$

Rule T-Let gives x a single type, which is not the whole story. Section 5.2 replaces it with the rule that lets one binding serve several types.

A recursive binding puts the name in scope inside its own definition. It is the only way to write a loop, since the language has no loop construct. The checker gives the name a type before it looks at the definition, then checks that the definition really has that type.

$$\frac{\Gamma, f : \tau_1 \vdash e_1 : \tau_1 \quad \Gamma, f : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \textbf{let rec}\ f = e_1\ \textbf{in}\ e_2 : \tau_2}\ \text{T-LetRec}$$

$$\frac{\rho, f \mapsto \langle f,\ \textbf{fun}\ x \to e,\ \rho \rangle \vdash e_2 \Downarrow v}{\rho \vdash \textbf{let rec}\ f = (\textbf{fun}\ x \to e)\ \textbf{in}\ e_2 \Downarrow v}\ \text{E-LetRec}$$

There is no mutual recursion, because neither of two functions that call each other is in scope in the other's definition.

3.10 Lists, cons and ranges

The list is the only compound data structure the language has. A list literal writes its elements between brackets separated by semicolons, and every element must have the same type. The elements

are evaluated from left to right.

$$\frac{\Gamma \vdash e_i : \tau \quad \text{for each } i}{\Gamma \vdash [\, e_1; \ldots; e_n \,] : \tau \ \mathsf{list}} \ \textsc{T-List} \qquad \frac{\rho \vdash e_i \Downarrow v_i \quad \text{for each } i}{\rho \vdash [\, e_1; \ldots; e_n \,] \Downarrow [v_1, \ldots, v_n]} \ \textsc{E-List}$$

The empty list is the case $n = 0$, and its element type is left free, so it can serve as a list of anything. Cons puts one value on the front of a list. It groups to the right and it builds a new list, leaving the old one available under its own name.

$$\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau \ \mathsf{list}}{\Gamma \vdash e_1 :: e_2 : \tau \ \mathsf{list}} \ \textsc{T-Cons} \qquad \frac{\rho \vdash e_1 \Downarrow v \quad \rho \vdash e_2 \Downarrow [v_1, \ldots, v_n]}{\rho \vdash e_1 :: e_2 \Downarrow [v, v_1, \ldots, v_n]} \ \textsc{E-Cons}$$

A range writes a run of consecutive integers. Both ends are included, and the range is empty when the upper bound is below the lower one, so `[1..3]` is `[1; 2; 3]` and `[3..1]` is `[]`.

$$\frac{\Gamma \vdash e_1 : \mathsf{int} \quad \Gamma \vdash e_2 : \mathsf{int}}{\Gamma \vdash [\, e_1 \,..\, e_2 \,] : \mathsf{int\ list}} \ \textsc{T-Range} \qquad \frac{\rho \vdash e_1 \Downarrow n_1 \quad \rho \vdash e_2 \Downarrow n_2}{\rho \vdash [\, e_1 \,..\, e_2 \,] \Downarrow [n_1, n_1 + 1, \ldots, n_2]} \ \textsc{E-Range}$$

3.11 Patterns

A pattern describes the shape of a value and names the parts worth keeping. There are seven forms: the wildcard, a variable, the three kinds of constant, the empty list and cons.

The judgement $p : \tau \dashv \Gamma$ says two things at once. It says which values the pattern can match, and it says what the pattern binds. The wildcard and the constants bind nothing, a variable matches anything and binds one name, and a cons pattern binds whatever its two parts bind.

$$\frac{}{\_ : \tau \dashv \cdot} \ \textsc{P-Wild} \qquad \frac{}{x : \tau \dashv x : \tau} \ \textsc{P-Var} \qquad \frac{}{n : \mathsf{int} \dashv \cdot} \ \textsc{P-Int} \qquad \frac{}{b : \mathsf{bool} \dashv \cdot} \ \textsc{P-Bool}$$

$$\frac{}{s : \mathsf{string} \dashv \cdot} \ \textsc{P-Str} \qquad \frac{}{[\,] : \tau \ \mathsf{list} \dashv \cdot} \ \textsc{P-Nil}$$

$$\frac{p_1 : \tau \dashv \Gamma_1 \quad p_2 : \tau \ \mathsf{list} \dashv \Gamma_2 \quad \mathrm{dom}(\Gamma_1) \cap \mathrm{dom}(\Gamma_2) = \emptyset}{p_1 :: p_2 : \tau \ \mathsf{list} \dashv \Gamma_1, \Gamma_2} \ \textsc{P-Cons}$$

The side condition on P-Cons rejects a pattern that binds the same name twice, since there would be no reason to prefer one of the two values it caught.

The judgement $p \triangleright v \Rightarrow \sigma$ says the pattern accepts the value and produces the bindings σ.

$$\frac{}{\_ \triangleright v \Rightarrow \cdot}\ \text{M-Wild} \qquad \frac{}{x \triangleright v \Rightarrow x \mapsto v}\ \text{M-Var} \qquad \frac{}{c \triangleright c \Rightarrow \cdot}\ \text{M-Const} \qquad \frac{}{[\,] \triangleright [\,] \Rightarrow \cdot}\ \text{M-Nil}$$

$$\frac{p_1 \triangleright v_1 \Rightarrow \sigma_1 \quad p_2 \triangleright [v_2, \ldots, v_n] \Rightarrow \sigma_2}{p_1 :: p_2 \triangleright [v_1, \ldots, v_n] \Rightarrow \sigma_1, \sigma_2}\ \text{M-Cons}$$

A pattern that no rule covers does not match, written $p \triangleright v \Rightarrow \bot$, and failure spreads from either part of a cons pattern to the whole.

3.12 Pattern matching

A `match` tries its cases from top to bottom and runs the first one that fits. It is how a program takes a list apart, and with the constant patterns it also serves as a multi-way conditional. Every case must produce the same type, every pattern must match the type of the scrutinee, and what a pattern binds is visible in that case's body and nowhere else.

$$\frac{\begin{array}{c}\Gamma \vdash e : \tau \\ p_i : \tau \dashv \Gamma_i \quad \text{for each } i \\ \Gamma, \Gamma_i \vdash e_i : \tau' \quad \text{for each } i \\ p_1, \ldots, p_n \text{ are exhaustive at } \tau\end{array}}{\Gamma \vdash \textbf{match } e \textbf{ with } p_1 \to e_1 \mid \cdots \mid p_n \to e_n : \tau'}\ \text{T-Match}$$

$$\frac{\begin{array}{c}\rho \vdash e \Downarrow v \\ p_j \triangleright v \Rightarrow \bot \quad \text{for each } j < i \\ p_i \triangleright v \Rightarrow \sigma \quad \rho, \sigma \vdash e_i \Downarrow v'\end{array}}{\rho \vdash \textbf{match } e \textbf{ with } p_1 \to e_1 \mid \cdots \mid p_n \to e_n \Downarrow v'}\ \text{E-Match}$$

The bindings σ come last, so a pattern variable hides an outer variable of the same name.

The last premise of T-Match is the exhaustiveness condition. A set of patterns is exhaustive at a type when every value of that type matches at least one of them. The checker decides this with Maranget's usefulness algorithm [7]: the patterns are exhaustive exactly when adding a further wildcard case would be useless, that is when no value escapes them all. The type says which sets of constructors are complete, so both booleans suffice for a boolean and `[]` together with `::` suffice for a list, while integers and strings are too many to cover without a wildcard.

The condition matters because Nomad has no exceptions. Without it a well typed program could still stop dead, and that was the one way it could.

3.13 The pipe operator

The pipe writes a call with the argument on the left, so a chain of steps reads in the order the data flows. Instead of `h (g (f x))`, write `x |> f |> g |> h`. It is sugar, rewritten by the parser, and

it groups to the left.

$$e_1 \mid\!> e_2 \quad \rightsquigarrow \quad e_2\ e_1$$

3.14 Statements and programs

A program is a list of statements, and each statement sees the bindings the earlier ones made. There is no separate entry point. The judgement $\Gamma \vdash \mathit{stmt} \dashv \Gamma'$ says the statement checks in Γ and leaves Γ' for the next one, and $\rho \vdash \mathit{stmt} \dashv \rho', v$ says it produces the value v and leaves ρ' behind.

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \textbf{let}\ x = e \dashv \Gamma, x : \tau}\ \text{T-SLet} \qquad \frac{\Gamma, f : \tau \vdash e : \tau}{\Gamma \vdash \textbf{let rec}\ f = e \dashv \Gamma, f : \tau}\ \text{T-SRec} \qquad \frac{\Gamma \vdash e : \tau}{\Gamma \vdash e \dashv \Gamma}\ \text{T-SExpr}$$

$$\frac{\rho \vdash e \Downarrow v}{\rho \vdash \textbf{let}\ x = e \dashv \rho, x \mapsto v,\ v}\ \text{E-SLet} \qquad \frac{\rho \vdash e \Downarrow v}{\rho \vdash e \dashv \rho,\ v}\ \text{E-SExpr}$$

The checker runs over the whole program before the evaluator runs over any of it, so a type error in the last statement stops the first one from running.

Every statement prints the value it produced, binding forms included. Values print as the source would write them, with strings in quotes and lists in brackets. A function prints as `<fun>`, since there is nothing useful to show.

4 Comprehensions

A comprehension writes a list by saying what its elements look like rather than by folding over another list. Of everything the language has, it is what shortens a Nomad program the most.

4.1 Notation

A comprehension is a body followed by one or more qualifiers, separated by a vertical bar. This is set-builder notation: read the bar as *such that*.

```
[x * x | x <- [1..10]] -- [1; 4; 9; ...; 100]
[x | x <- xs, x > 0] -- the positive elements
[y | x <- xs, let y = f x, y > 0] -- name a value, then test it
[x * 10 + y | x <- [1; 2], y <- [3; 4]] -- [13; 14; 23; 24]
[x | r <- xss, x <- r] -- flatten a list of lists
[[0 | j <- [1..n]] | i <- [1..n]] -- an n by n grid of zeros
[x | h :: t <- xss] -- heads of the non-empty lists
```

There are three kinds of qualifier. A *generator* $p \leftarrow e$ draws values from a list. A *guard* is any boolean expression. A `let` names a value, and it takes parameters like an ordinary binding.

4.2 What the qualifiers mean

The qualifiers run left to right and nest, so the leftmost generator is the outer loop and the rightmost varies fastest. A later qualifier sees every name an earlier one bound, which is what makes `[.. | i <- [1..n], j <- [1..i]]` enumerate a triangle. A guard skips the qualifiers after it for the value in hand, and a `let` extends the scope of the qualifiers after it and of the body.

Two decisions are worth stating. The language is strict, so a comprehension runs to completion and there are no infinite ranges. And a generator pattern may be refutable: a value it does not match is skipped rather than being an error. That is what makes `[x | h :: t <- xss]` pick out the heads of the non-empty lists, and it is the one place where the exhaustiveness condition of T-Match deliberately does not apply.

4.3 Static semantics

Qualifiers are typed in order, each adding to the environment the next one sees. The judgement $\Gamma \vdash q \dashv \Gamma'$ says qualifier q checks in Γ and leaves Γ'.

$$\frac{\Gamma \vdash e : \tau \ \mathsf{list} \quad p : \tau \dashv \Gamma_p}{\Gamma \vdash p \leftarrow e \dashv \Gamma, \Gamma_p}\ \text{Q-Gen} \qquad \frac{\Gamma \vdash e : \mathsf{bool}}{\Gamma \vdash e \dashv \Gamma}\ \text{Q-Guard} \qquad \frac{\Gamma \vdash e : \tau}{\Gamma \vdash \mathbf{let}\ x = e \dashv \Gamma, x : \tau}\ \text{Q-Let}$$

The comprehension types its body in the environment the last qualifier leaves, and the whole form is a list of whatever the body produces.

$$\frac{\Gamma_0 = \Gamma \quad \Gamma_{i-1} \vdash q_i \dashv \Gamma_i \quad \text{for each } i \in \{1, \ldots, k\} \quad \Gamma_k \vdash e : \tau}{\Gamma \vdash [\, e \mid q_1, \ldots, q_k \,] : \tau \ \mathsf{list}}\ \text{T-Comp}$$

4.4 Dynamic semantics

The judgement $\rho \vdash \langle \vec{q} \mid e \rangle \Downarrow [v_1, \ldots, v_n]$ says that running the qualifiers $\vec{q}$ in ρ and evaluating e each time they succeed produces exactly that list, in that order.

When no qualifiers are left, the body is evaluated and gives one element.

$$\frac{\rho \vdash e \Downarrow v}{\rho \vdash \langle \cdot \mid e \rangle \Downarrow [v]}\ \text{C-Body}$$

A generator evaluates its list once and then walks it, concatenating what each value contributes. Write $\rho, p \triangleright u \vdash \langle \vec{q} \mid e \rangle \Downarrow L$ for what a single value u contributes when the generator's pattern is p. A

value the pattern accepts contributes whatever the remaining qualifiers produce under the bindings it made, and a value the pattern rejects contributes nothing.

$$\frac{p \triangleright u \Rightarrow \sigma \quad \rho, \sigma \vdash \langle \vec{q} \mid e \rangle \Downarrow L}{\rho, p \triangleright u \vdash \langle \vec{q} \mid e \rangle \Downarrow L}\ \text{C-Hit} \qquad \frac{p \triangleright u \Rightarrow \bot}{\rho, p \triangleright u \vdash \langle \vec{q} \mid e \rangle \Downarrow [\,]}\ \text{C-Miss}$$

The generator is then the concatenation, in order, of what each of its values contributed.

$$\frac{\rho \vdash e_g \Downarrow [u_1, \ldots, u_m] \quad \rho, p \triangleright u_i \vdash \langle \vec{q} \mid e \rangle \Downarrow L_i \quad \text{for each } i}{\rho \vdash \langle (p \leftarrow e_g),\ \vec{q} \mid e \rangle \Downarrow L_1 +\!\!+ \cdots +\!\!+ L_m}\ \text{C-Gen}$$

A guard either carries on or contributes nothing, and a `let` extends the environment.

$$\frac{\rho \vdash e_b \Downarrow \textbf{true} \quad \rho \vdash \langle \vec{q} \mid e \rangle \Downarrow L}{\rho \vdash \langle e_b,\ \vec{q} \mid e \rangle \Downarrow L}\ \text{C-GuardT} \qquad \frac{\rho \vdash e_b \Downarrow \textbf{false}}{\rho \vdash \langle e_b,\ \vec{q} \mid e \rangle \Downarrow [\,]}\ \text{C-GuardF}$$

$$\frac{\rho \vdash e_l \Downarrow v \quad \rho, x \mapsto v \vdash \langle \vec{q} \mid e \rangle \Downarrow L}{\rho \vdash \langle (\textbf{let } x = e_l),\ \vec{q} \mid e \rangle \Downarrow L}\ \text{C-Let}$$

The comprehension itself is then just that list.

$$\frac{\rho \vdash \langle q_1, \ldots, q_k \mid e \rangle \Downarrow L}{\rho \vdash [\, e \mid q_1, \ldots, q_k \,] \Downarrow L}\ \text{E-Comp}$$

Comprehensions came to functional programming through NPL and then Miranda, and the translation Wadler gives [8] is the one every implementation still uses. Nomad does not desugar, because there is no standard library to desugar into and because a construct of its own gives errors that point at what the programmer wrote.

5 Type inference

A Nomad program carries no annotations, so the checker finds every type by itself. It uses the standard method: give the unknowns names, collect the constraints the program forces on those names, and solve them by unification. Pierce gives the full treatment [2], and the Cornell textbook works the same algorithm through in OCaml [3].

5.1 Substitutions and unification

A type variable α stands for a type nobody has pinned down. The checker makes a fresh one whenever it meets an unknown, such as a function parameter or the element type of an empty list.

A substitution is a finite map from type variables to types, and composition $S_2 \circ S_1$ applies S_1 first. One substitution is threaded through the whole traversal, refined at every step.

Unification takes two types and returns the most general substitution making them equal, or fails. The algorithm is Robinson's [4]. Identical types unify under the empty substitution, a variable unifies with any type by pointing at it, two function types unify when their arguments and then their results do, and two list types unify when their element types do.

One case needs care. A variable must not unify with a type that contains it, since binding α to $\alpha \to \mathsf{int}$ would describe a type with no finite form. The occurs check rejects it, which is why `fun x -> x x` does not check.

5.2 Generalisation

Rules T-Var and T-Let were stated for plain types, which would give each binding exactly one type for its whole body. Full Hindley-Milner inference does better [5, 6]. The environment records a *scheme* $\forall \alpha_1 \ldots \alpha_n.\ \tau$, and a `let` binding quantifies the variables of its type that the environment does not already constrain.

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \sigma = \forall \vec{\alpha}.\, \tau_1 \quad \vec{\alpha} = \mathrm{fv}(\tau_1) \setminus \mathrm{fv}(\Gamma) \quad \Gamma, x : \sigma \vdash e_2 : \tau_2}{\Gamma \vdash \textbf{let}\ x = e_1\ \textbf{in}\ e_2 : \tau_2}\ \text{T-Let'}$$

$$\frac{x : \forall \vec{\alpha}.\, \tau \in \Gamma}{\Gamma \vdash x : \tau[\vec{\alpha} := \vec{\beta}]}\ \text{T-Var'}$$

In T-Var' the $\vec{\beta}$ are fresh, so every use of the name gets its own copy. This is what lets one definition serve several types, and without it no standard library could exist, because a single `map` would work on lists of integers or lists of strings but never both in one program. A top-level binding and a comprehension `let` generalise the same way.

Function parameters and pattern variables do not generalise. Their types are fixed by the caller, not by the binding, so `fun f -> if f true then f 1 else 0` is rejected, as it should be.

Nomad needs no value restriction yet, because nothing in the language mutates. Once references arrive, generalising the type of an expression that allocates one would be unsound, and the restriction becomes necessary.

6 Beyond this course

These are the features planned after borrow checking, shortest first.

A conditional expression. The C form is sugar for `if` and expands at parse time.

```
x > 0 ? 1 : -1
```

Parallel generators. A second bar walks two lists in step rather than taking their product, as GHC's parallel comprehensions do. Writing that by hand needs a zip and a map.

```
[x + y | x <- xs | y <- ys]
```

Comprehensions as functions. A hole in place of a generator's source gives a function of that list. Holes are marked, because guessing which free names are parameters cannot be done reliably.

```
[x * 2 | x <- _] -- a function taking one list
```

Collectors. A name before the bracket lets the same notation sum, count or fold instead of always building a list, following the generator and accumulator split of SRFI 42 [9].

```
sum[x * x | x <- [1..n]] -- no list is ever built
count[x | x <- xs, x = target]
```

A standard library. Nomad has no library, so every program writes its own `concat` and `length`. Generalisation is in place, which was the prerequisite, and the list and string functions come next.

Functional arrays. Lists give linear-time access, which rules out most of the algorithms a contest problem asks for. Nomad needs an array with constant-time access that still behaves as a value: updating it must not copy it, and must not let an old name see the new contents. The array also needs native operations and a short way to write one down. Which technique to adopt is the open question, whether an affine type that makes the old array inaccessible after an update, a persistent structure with a fast recent version, or an analysis that proves the copy unnecessary. This is where borrow checking pays off.

Matrices. Native two-dimensional structures with native operations, built on the arrays above. Errington argues that the index is the wrong interface to a grid and shows what grid code looks like when the program never mentions one [10]: focus a cell, keep its neighbours in reach, and a rule that reads them needs no arithmetic. That also answers a limit of comprehensions, which compute each element independently and so cannot express a table whose cells depend on earlier cells.

Input and output. Reading input should take one line. C++ manages this with stream extraction, where the values read take the types the variables already have.

```
cin >> x >> y -- C++, for comparison
```

OCaml wraps input in a channel and asks the programmer to learn a library, and Python sits in between. Nomad should read the values a program needs, at the types it needs, without naming a channel or a parser. Doing this under full inference is an open problem, since the type of what is read has to come from how it is used.

References

[1] Graham Hutton and Erik Meijer. *Monadic Parser Combinators*. Technical Report NOTTCS-TR-96-4, Department of Computer Science, University of Nottingham, 1996. https://people.cs.nott.ac.uk/pszgmh/monparsing.pdf

[2] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.

[3] Michael R. Clarkson et al. *OCaml Programming: Correct + Efficient + Beautiful*. Cornell University, CS 3110. https://cs3110.github.io/textbook/cover.html

[4] J. A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, 1965.

[5] Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3):348–375, 1978.

[6] Luis Damas and Robin Milner. Principal type-schemes for functional programs. In *Proceedings of the 9th ACM Symposium on Principles of Programming Languages*, pages 207–212, 1982.

[7] Luc Maranget. Warnings for pattern matching. *Journal of Functional Programming*, 17(3):387–421, 2007.

[8] Philip Wadler. List comprehensions. Chapter 7 of Simon L. Peyton Jones, *The Implementation of Functional Programming Languages*. Prentice Hall, 1987.

[9] Sebastian Egner. *SRFI 42: Eager Comprehensions*. Scheme Requests for Implementation, 2003. https://srfi.schemers.org/srfi-42/srfi-42.html

[10] Jacob Errington. *Grids without indices*. 2025. https://jerrington.me/posts/2025-12-23-grids-without-indices.html